Cloud-ifying MetGenC — Architecture Options and Recommendations

13 October 2026

1. Background

MetGenC is the command-line tool NSIDC operators use to create granule-level metadata (UMM-G) for ingest into the Earthdata Cloud (via Cumulus) archive in AWS. For each granule in a collection, it:

1. reads the science data files — plus premet and spatial files, where the operator supplies them — and generates UMM-G metadata for NASA's Common Metadata Repository (CMR);
2. copies the data files and the UMM-G metadata to the Cumulus staging bucket in S3; and
3. posts a Cloud Notification Mechanism (CNM) message to a Kinesis stream, which triggers Cumulus to ingest the granule.

Today an operator runs MetGenC by hand on an on-prem NSIDC VM. The operator places the collection's files on NSIDC's network storage, writes a `.ini` configuration file for the collection, loads AWS credentials for the target Cumulus environment (UAT or production), and runs `metgenc process`.

MetGenC was built for that workflow, and it assumes its input is on a local disk. That assumption breaks once data is delivered in the cloud, as Earthdatapub CUE will do (§4). §2 lists this and MetGenC's other limitations in the cloud.

This document sets out the options for running MetGenC on cloud-resident data and recommends one. The plan of this document:

- §2 lists MetGenC's limitations in the cloud and the updates they call for;
- §3 states the requirements every option must meet;
- §4 describes the options and the recommendation;
- §5 compares the options, explains the recommendation, and considers a container image; and
- the Appendix lists open questions, none of which block the decision.

2. MetGenC Limitations

MetGenC already writes to AWS — it copies files to S3 and posts to Kinesis — but it reads its input only from disk, and its credentials, logging, and host all assume the on-prem VM.

Limitation today	Update needed	MetGenC code change?
Reads input files only from a local or network-mounted disk	Bring the data to a disk (Option A), or have MetGenC read S3 directly (Option B)	Option B only
Every file passes through the machine running MetGenC, to compute its checksum and to upload it	Have S3 copy the file and compute its checksum itself, where the provider's access model allows	Option B only
Uses one set of AWS credentials for everything	Accept a second set for the provider's bucket, for providers whose access model requires it	Option B only, and only for those providers
AWS access uses long-lived keys stored on the VM; Earthdata Login credentials are set in the VM's login profile	An IAM role for the EC2 instance; Earthdata Login credentials from AWS Secrets Manager or SSM Parameter Store	No
Logs are written to a file in an on-prem directory	Ship the log file to CloudWatch and set alarms on it	No
Its only production host is the on-prem VM	An EC2 host built from the NGAP AMI with Terraform	No

The last three updates are deployment work that every option needs. The first three are where the options in §4 differ.

3. Requirements

To run MetGenC in the cloud, whether from Earthdatapub CUE or other sources, we have the following must-have and nice-to-have requirements:

Requirement	
Read source data from a provider-owned S3 bucket in a different AWS account from NSIDC's destination (CUE)	**Must**
Operator-driven runs, as today	**Must**
One codebase that also keeps the on-prem workflow working, indefinitely	**Must**
Run on EC2, built from the NGAP AMI with Terraform	**Must**
Logs streamable to CloudWatch so that alerts can be created	**Must**
Event-driven (unattended) runs	Nice to have
Container image	Nice to have — may cost less to run (§5)

4. Options

Earthdatapub CUE will deliver data to NSIDC in **provider-owned S3 buckets, in AWS accounts other than NSIDC's**. MetGenC today can only read files from a local disk. The decision to make is how much development we undertake to close that gap:

- **Option A — run today's MetGenC, unchanged, on an EC2 instance in AWS.** Something other than MetGenC brings the data to it: either a copy onto the instance's disk (**A1**), or a mount that makes the provider's bucket look like a local directory (**A2**).
- **Option B — make MetGenC read from S3 itself.** Everything in Option A, plus code changes so that MetGenC lists, reads, and stages data in S3 directly.

Recommendation: fund Option B. Option A works, and is an acceptable fallback if funding allows nothing more, but it is sub-optimal for the reasons described below. All of Option A's work except its copy or mount step is reused by Option B, so funding A now does not foreclose B later.

5. Options compared

	A1 — EC2 + copy	**A2 — EC2 + mount**	**B — S3-native MetGenC**
How data reaches MetGenC	Operator copies it from the provider bucket to the instance's disk, then runs MetGenC	Provider bucket is mounted read-only; MetGenC reads it as ordinary files	MetGenC reads the provider bucket directly
MetGenC code changes	None	None	Yes
Meets the CUE requirement	By workaround — a copy tool reads S3, not MetGenC	By workaround — the mount reads S3, not MetGenC	Yes
Extra operator work per run	Copy, size the disk, clean up	None once mounted	None
Proven?	Partly — one granule, spring 2026	No — needs a short trial	No — needs development
Times each data file crosses the network (affects run time)	2	3	0 – 2
Moves MetGenC toward unattended runs	No	No	Partly
Relative effort	One M and five S tasks	Same as A1	A, plus one L and three to four M

Effort sizes are relative — S(mall), M(edium), L(arge) — and are engineering judgment, not hour estimates.

- **Today**, data already in S3 is copied to the on-prem host `nusnow`, paid for as S3 internet egress.
- **All three options run in us-west-2**, the same region as the provider buckets, and AWS does not charge for moving data between S3 and EC2 within a region. **Every option removes today's egress charge** — provided the instance reaches S3 through an S3 *gateway endpoint*.

Option A's shortfall is therefore mainly **operational**:

- **A1** copies every granule in full before processing. The operator must run the copy with the provider's credentials, size the disk for the batch, and clean up afterwards. Every run waits for the copy.
- **A2** removes the copy step and the disk sizing, but it is untried on the NGAP AMI, and it reads every file in full at least twice through the mount.
- **Neither** can have S3 perform the copy to NSIDC's bucket server-side, and neither moves MetGenC any closer to unattended operation.

Why Option B is recommended:

- **It meets the CUE requirement as stated.** MetGenC itself reads the provider's bucket, with no copy tool or mount standing in for it.
- **It removes the per-run operator work.** There is no copy to run, no disk to size or clean up, and no mount to maintain. The operator puts an `s3://` location in the `.ini` file where a directory path goes today.
- **It moves the least data.** When the provider grants access to NSIDC's destination identity, S3 copies each file to the Cumulus staging bucket and computes its checksum itself, so most data never passes through the instance.
- **It keeps on-prem working.** Local paths keep working alongside `s3://` locations, so the on-prem workflow is unchanged and there is still one codebase.
- **It is the foundation for what comes next.** Unattended runs and a container image (below) are both far simpler when MetGenC reads S3 directly than when data must first be put on a disk.
- **It builds on Option A rather than replacing it.** Option A's host, credentials, and logging work all carries over.

A container image is cheap to provide, and could lower running cost:

- **Cheap to provide.** The image packages the same MetGenC that is installed on the VM and on EC2: one short build file, plus a CI step that publishes the image with each release. MetGenC's compiled dependencies install as pre-built packages, so the image needs no extra system libraries. MetGenC already logs to the console, which the container platform can send straight to CloudWatch. The image is an addition, not a replacement: the on-prem VM does not use containers and keeps the existing package.
- **Could cost less to run.** An EC2 instance is billed for every hour it is running, whether MetGenC is processing or idle. A container can instead run on AWS Fargate, which bills only while a run is in progress. Operator-driven runs are intermittent, so most of an always-on instance's hours would be idle.
- **Conditions.** The saving is measured against an instance left running; stopping the instance between runs captures most of it, at the cost of someone starting and stopping it. Fargate's hourly rate is higher than EC2's for the same capacity, so continuous processing would cost more, not less. The container pairs naturally with Option B; Options A1 and A2 depend on a disk copy or a mount, which are harder to arrange in a short-lived container. Running on Fargate rather than EC2 would also need agreement, since EC2 is the stated requirement (§3).

Appendix: Open items — none block this decision

Unknown	Effect
How each provider grants access: a bucket policy naming NSIDC, or NSIDC assuming a role in the provider's account	Both are planned for. It determines how much of Option B's data-movement saving is realized.
Verify NGAP network reaches S3 through a gateway endpoint	Determines whether moving data between S3 and EC2 incurs a charge.
Where operator-supplied premet/spatial files will live in the cloud	Option B handles any answer; Option A needs an Ops decision.
Whether the S3 mount works on the NGAP AMI	Decides only between A1 and A2.
Whether NGAP permits running MetGenC on AWS Fargate	Decides whether a container's running-cost saving is available.